Departamento de Informática

Bacharelado em Ciência da Computação

CI1212 Arquitetura de Computadores

Construindo o Processador

MIPS Multiciclo

PE-2020
Prof. Eduardo Todt
Versão 202007221921

MIPS Multiciclo

Objetivos

- Entender problema de desempenho do MIPS monociclo
- Conhecer implementação MIPS Multiciclo
- Comparar desempenho

MIPS monociclo

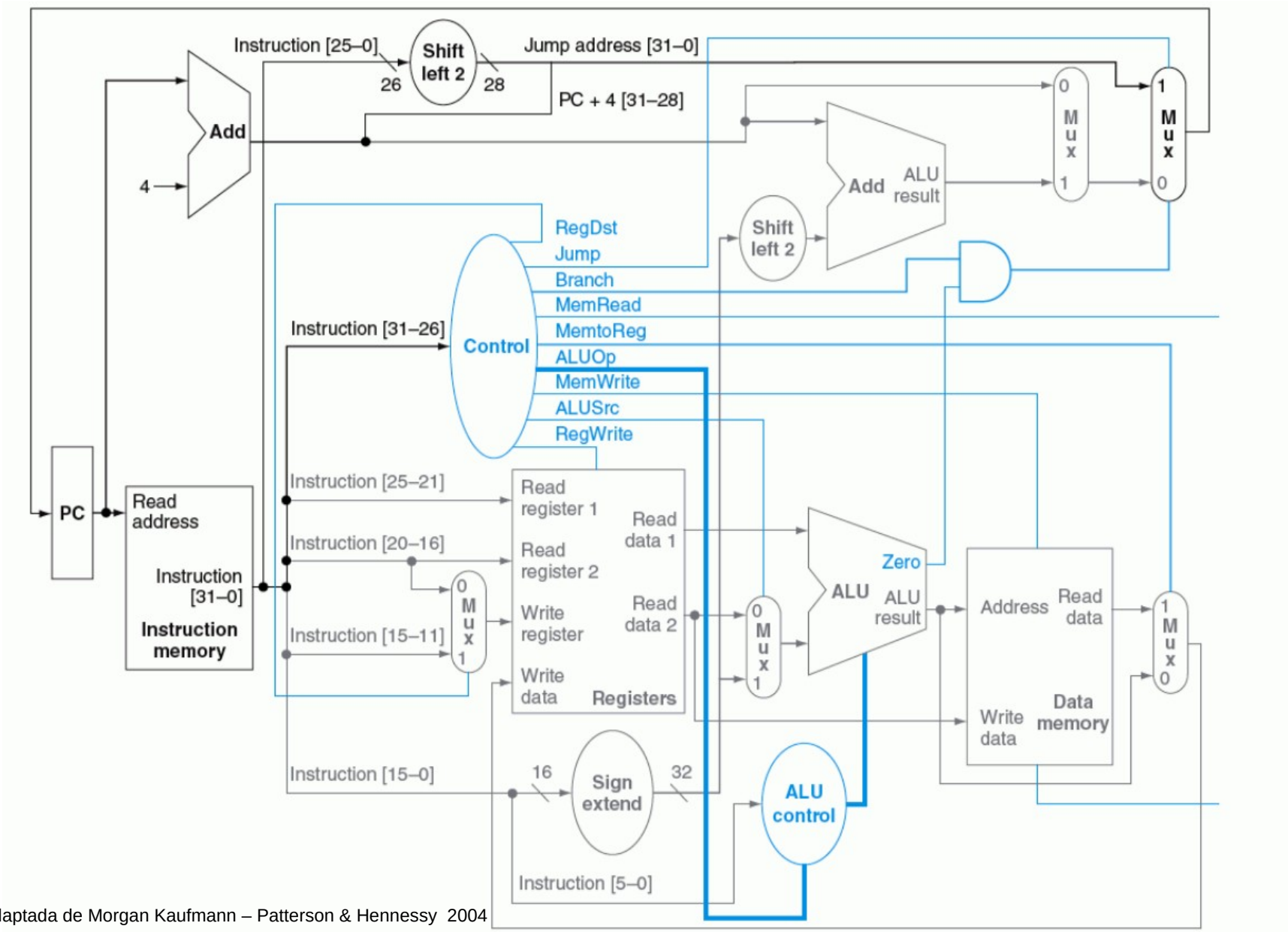
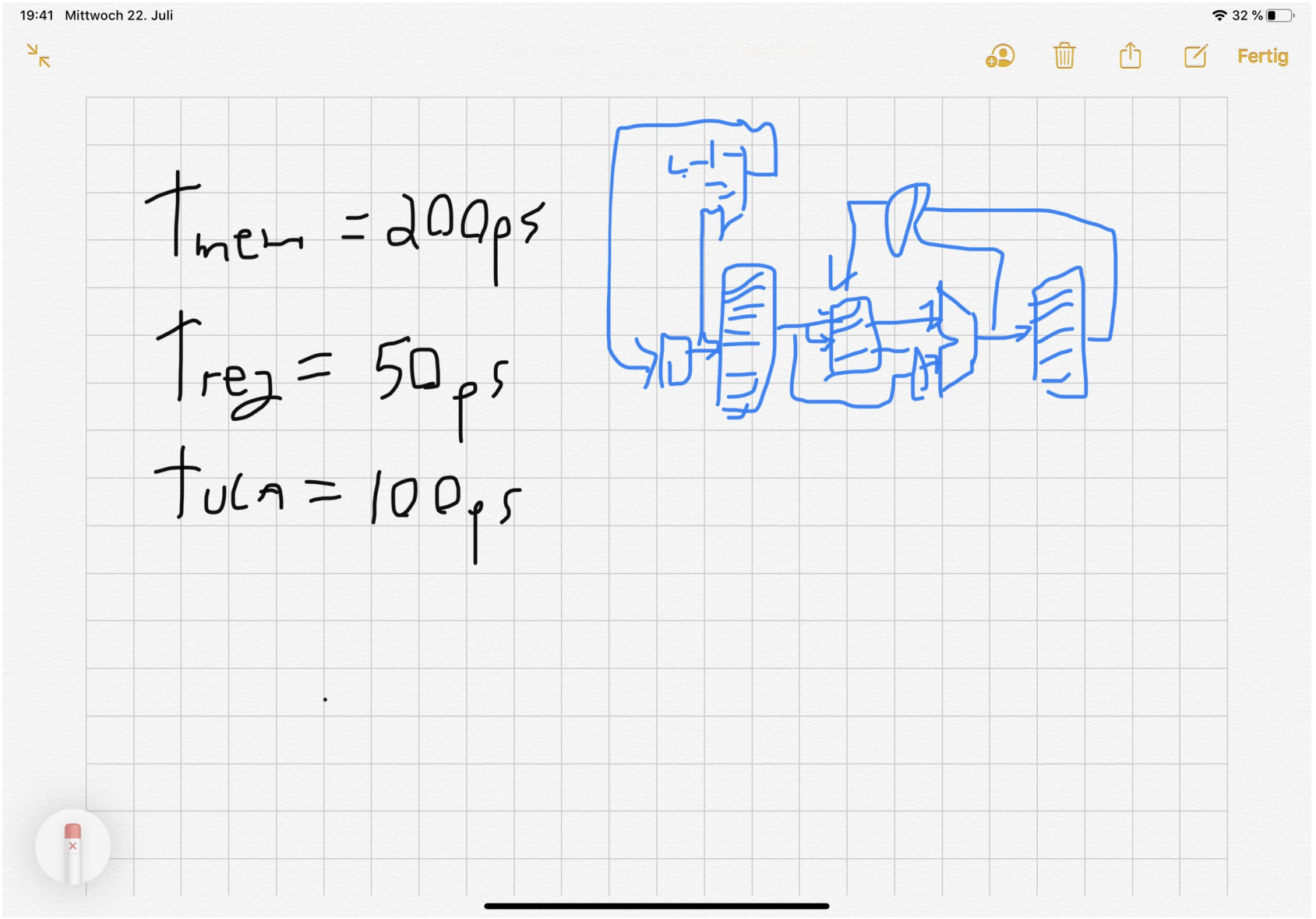
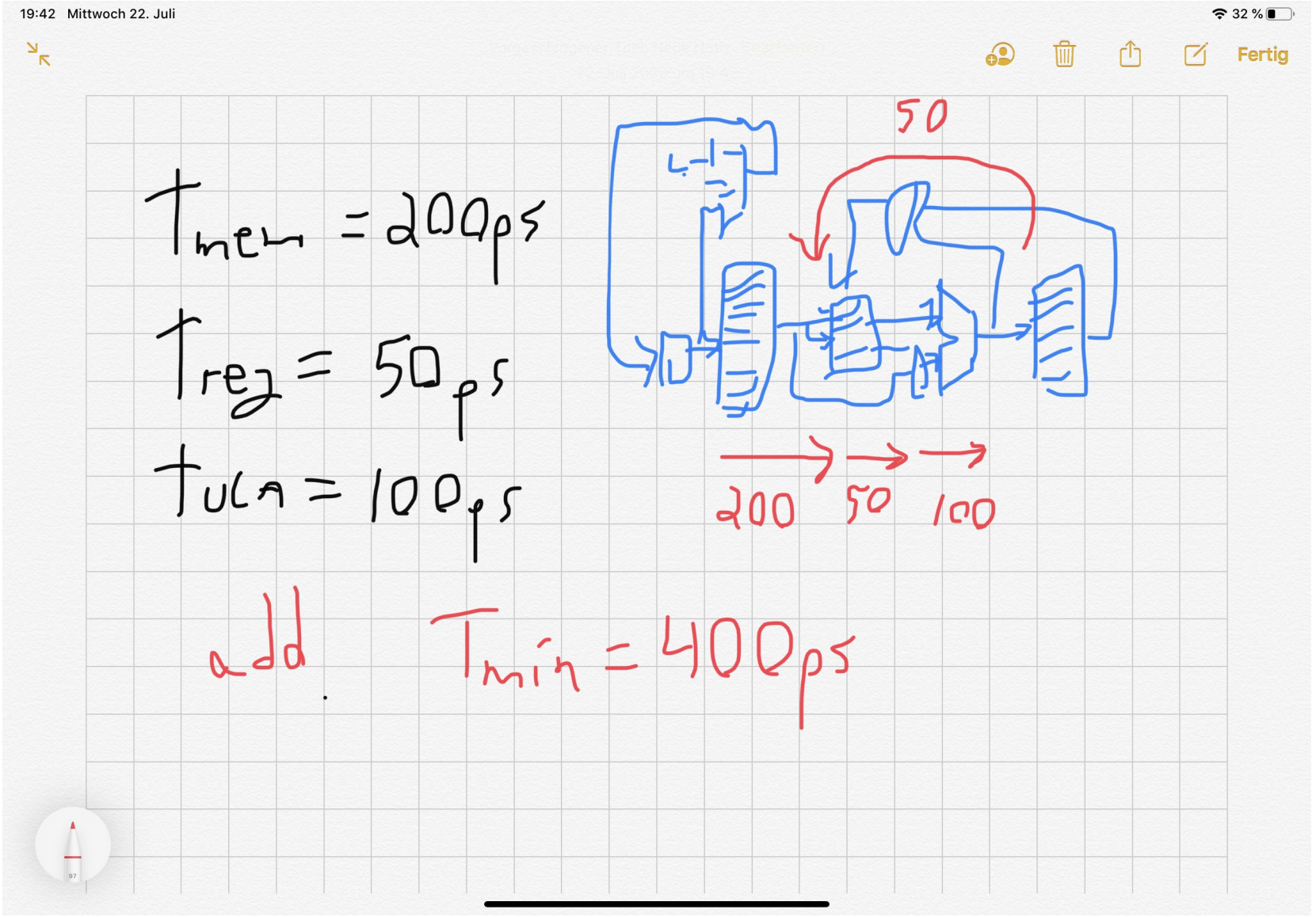


Figura adaptada de Morgan Kaufmann – Patterson & Hennessy 2004

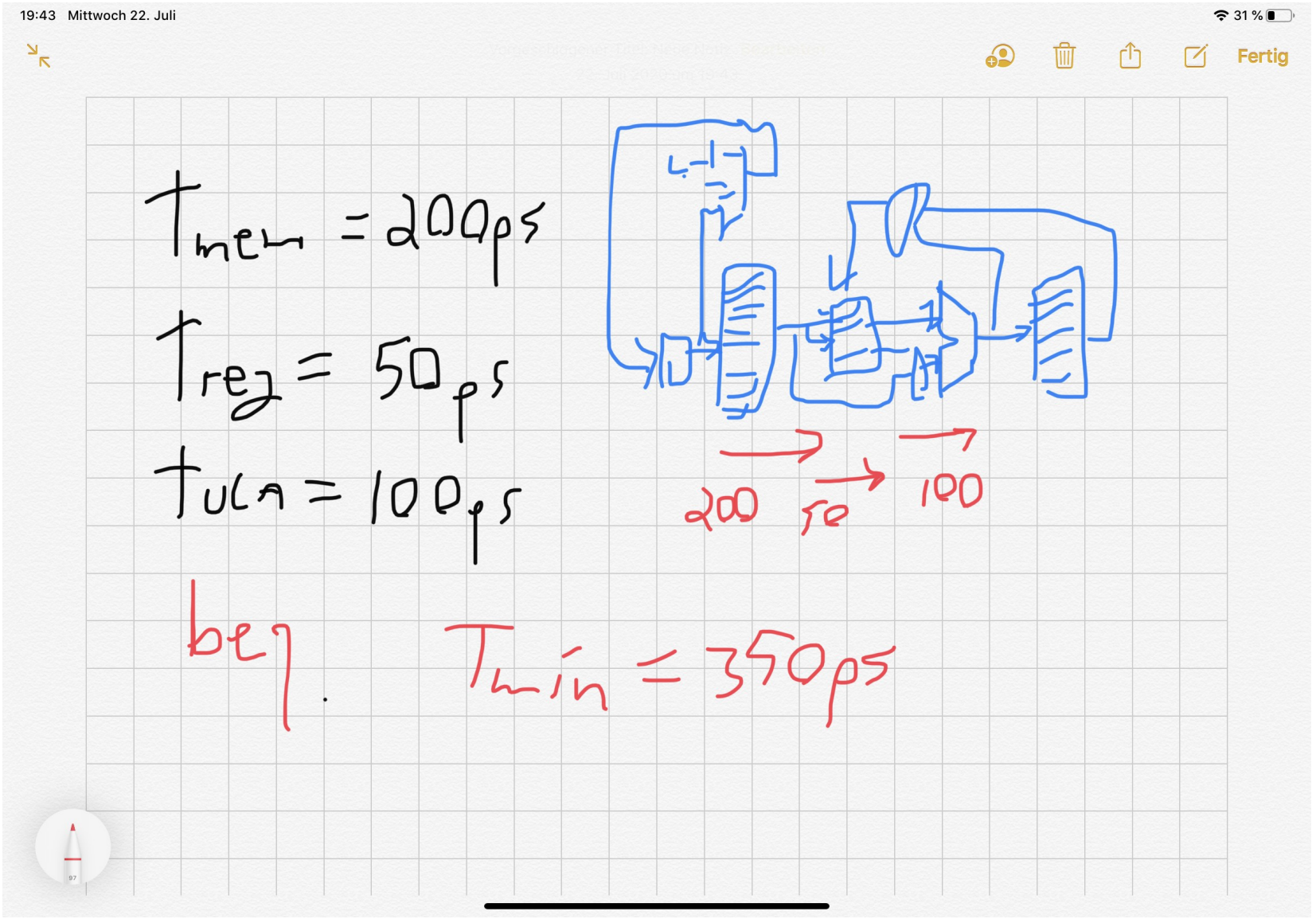
Período mínimo clock



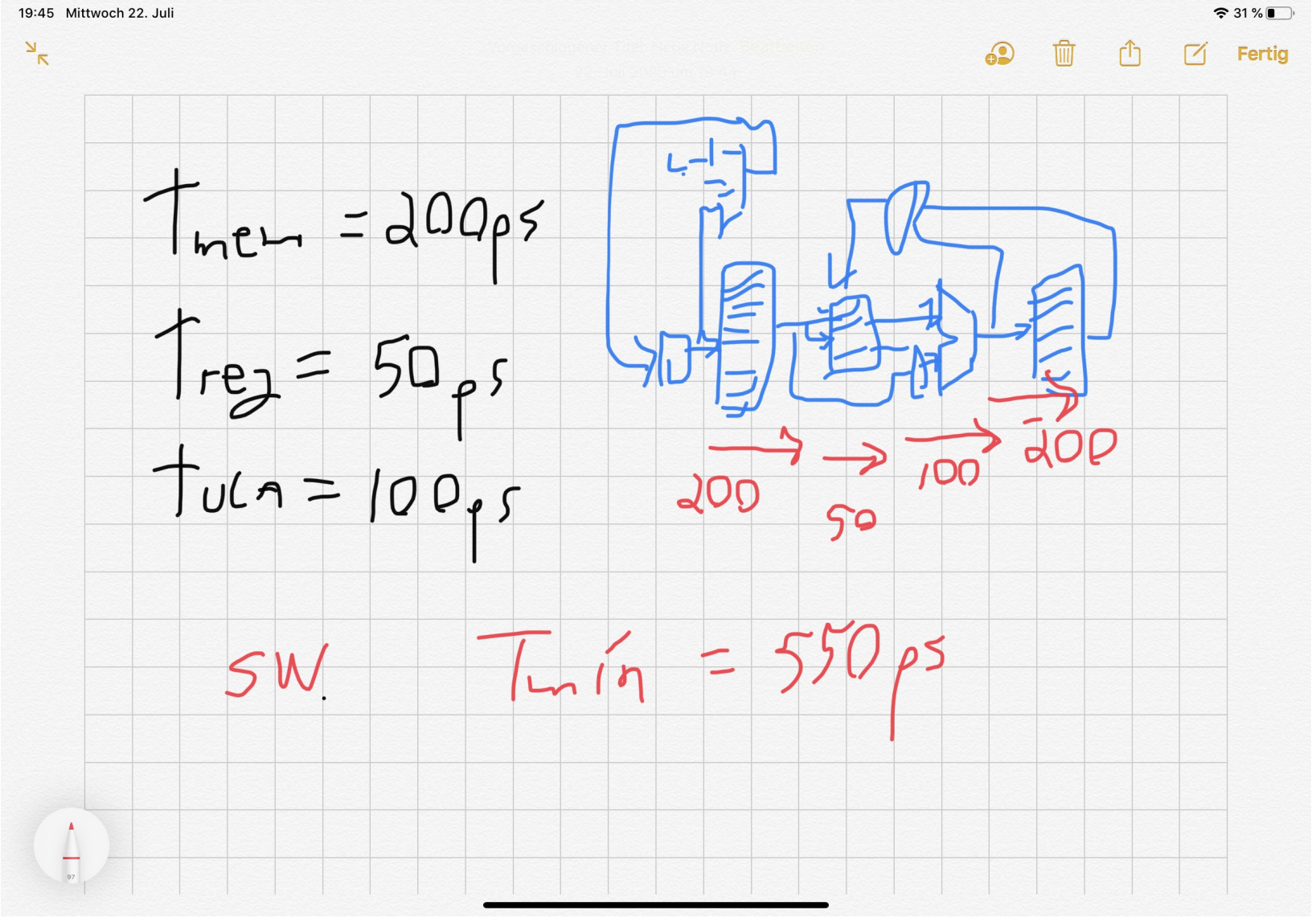
Período mínimo clock



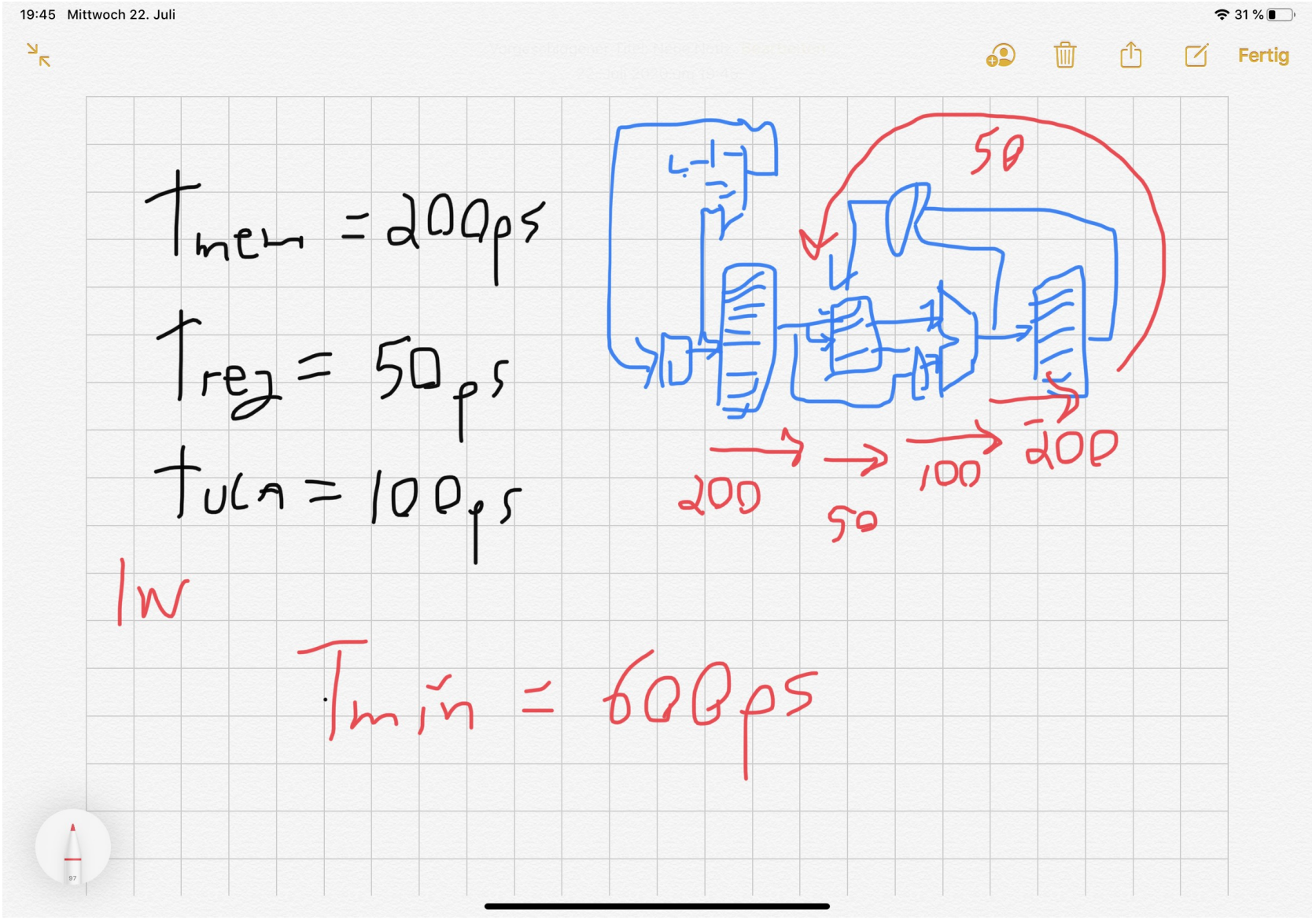
Período mínimo clock



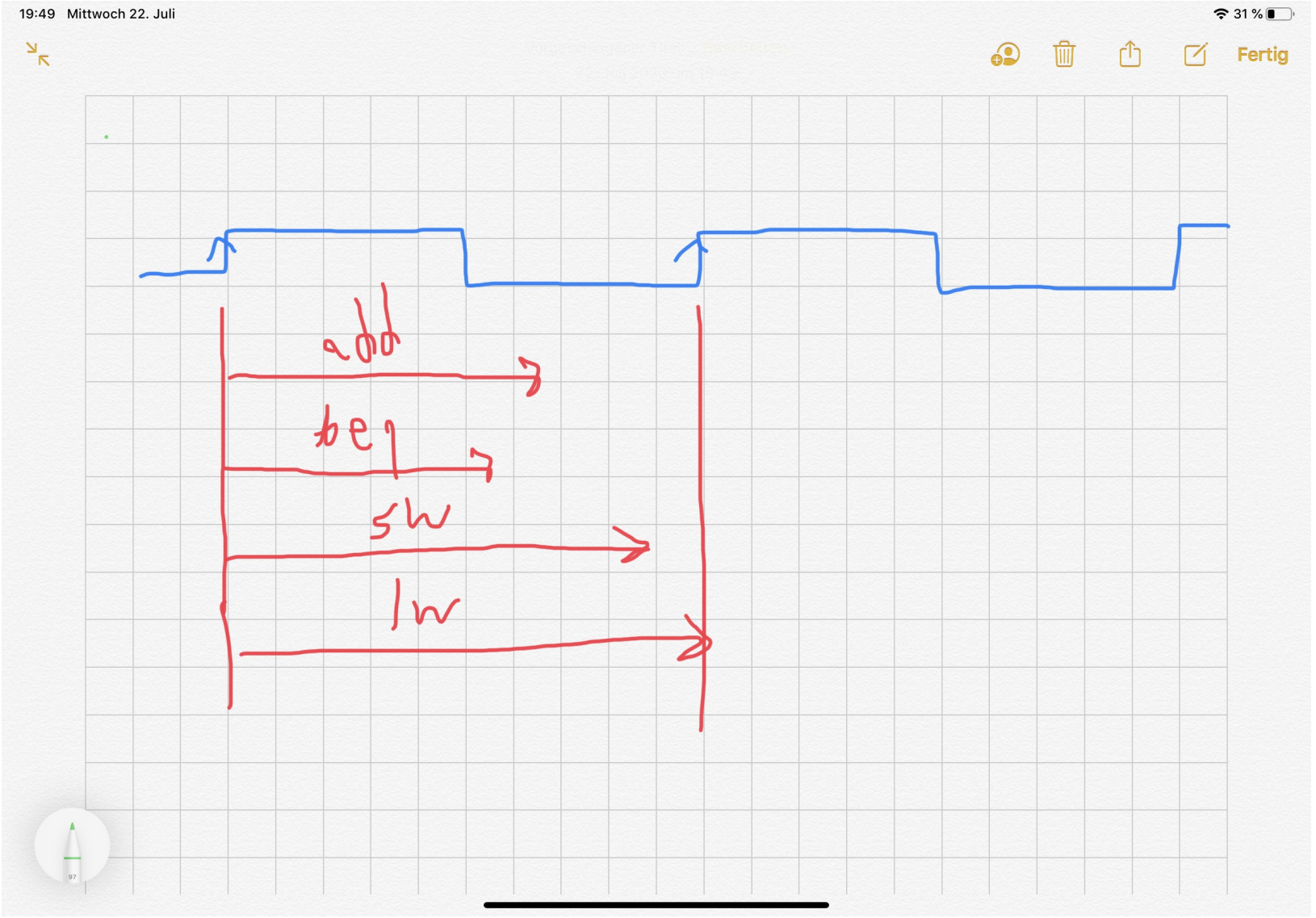
Período mínimo clock



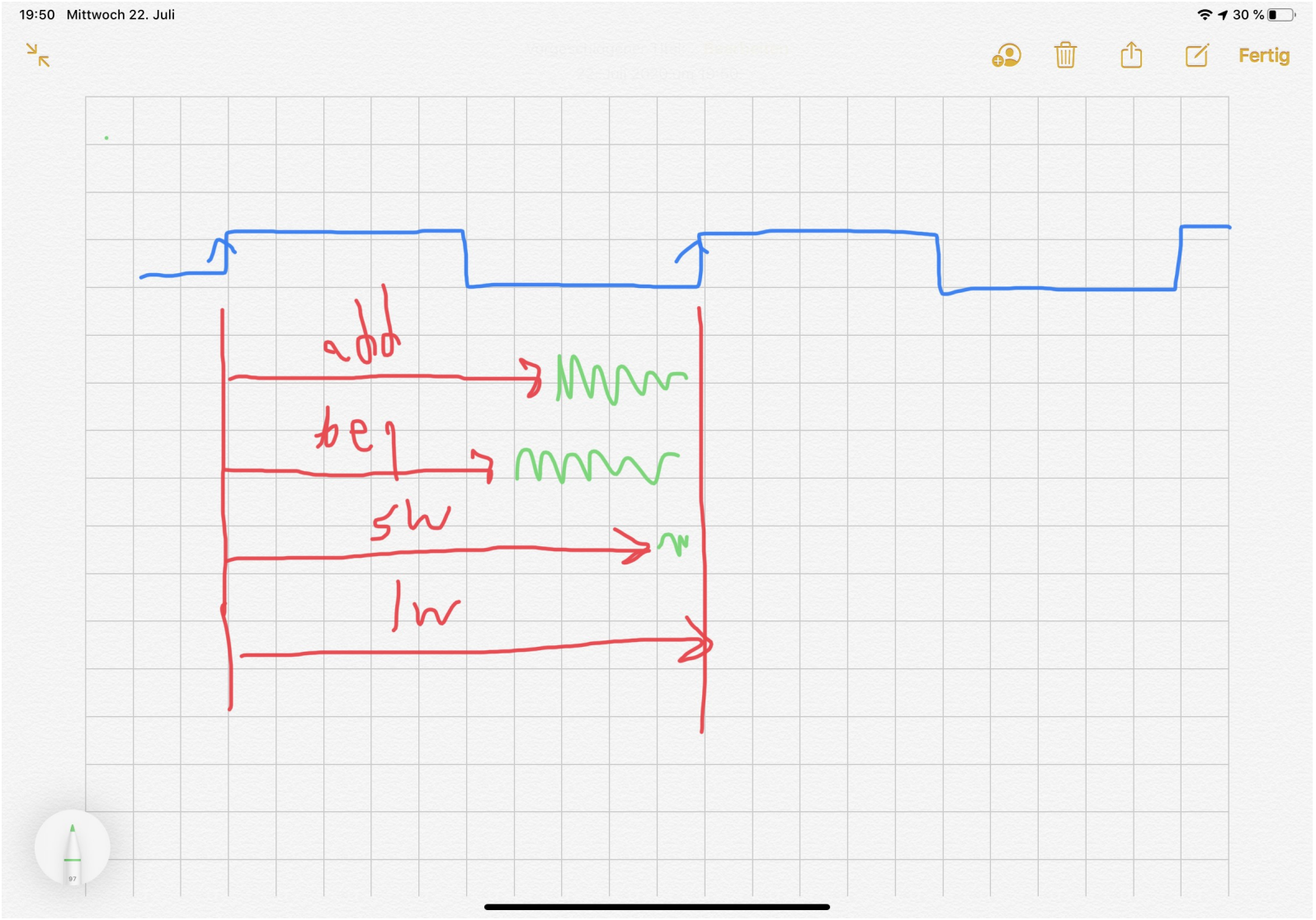
Período mínimo clock



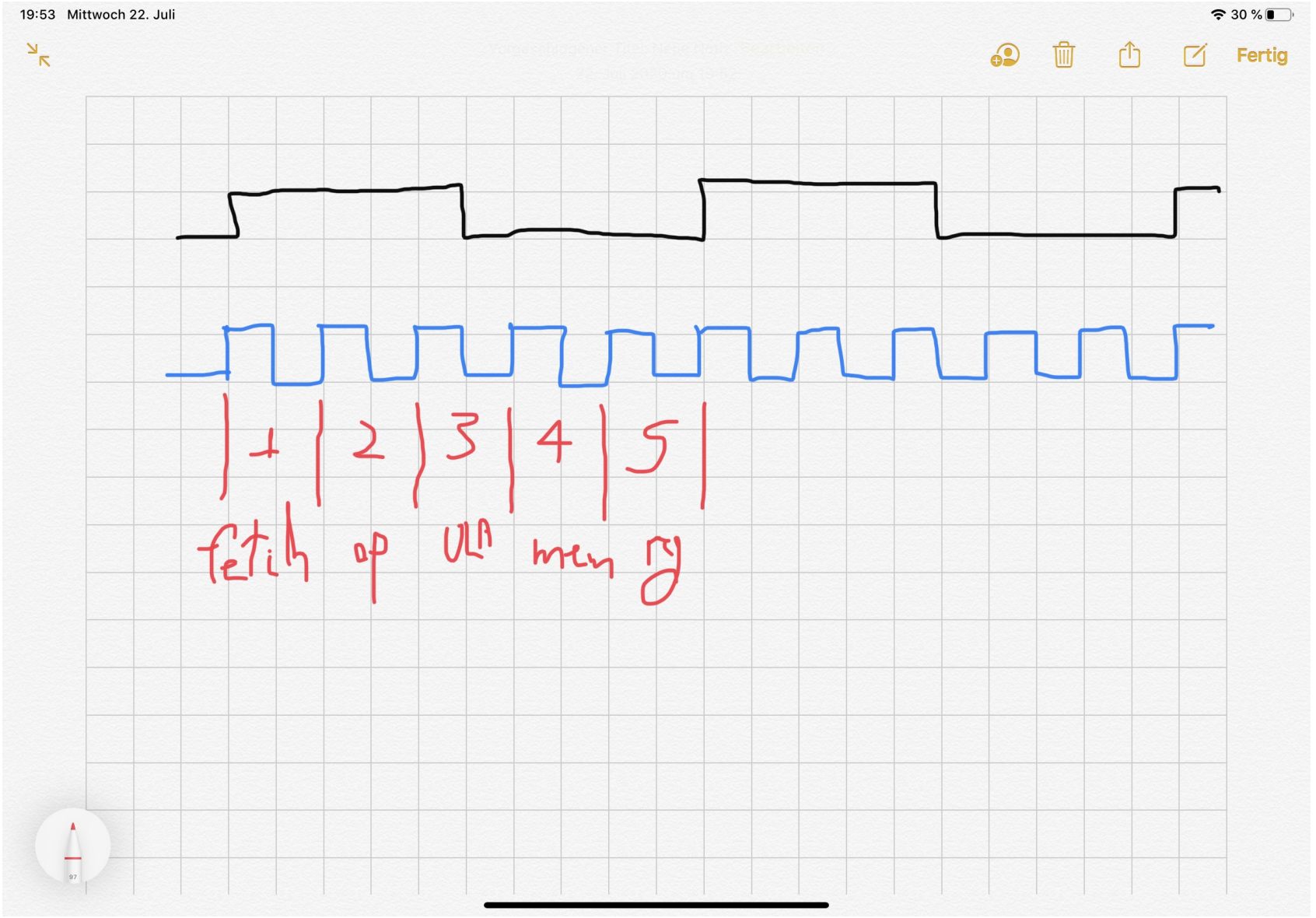
T_{min} deve atender pior caso



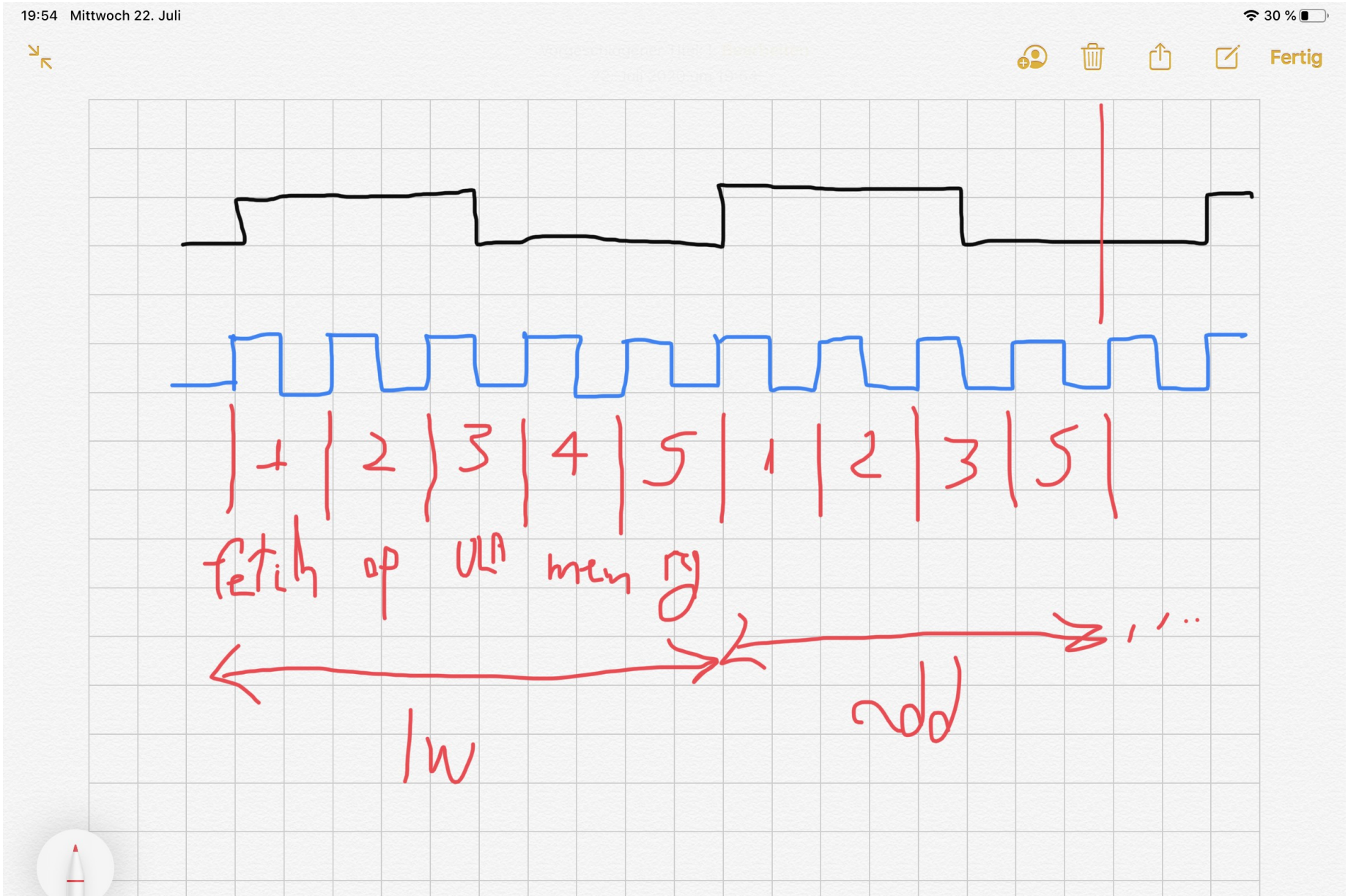
Tempo desperdiçado



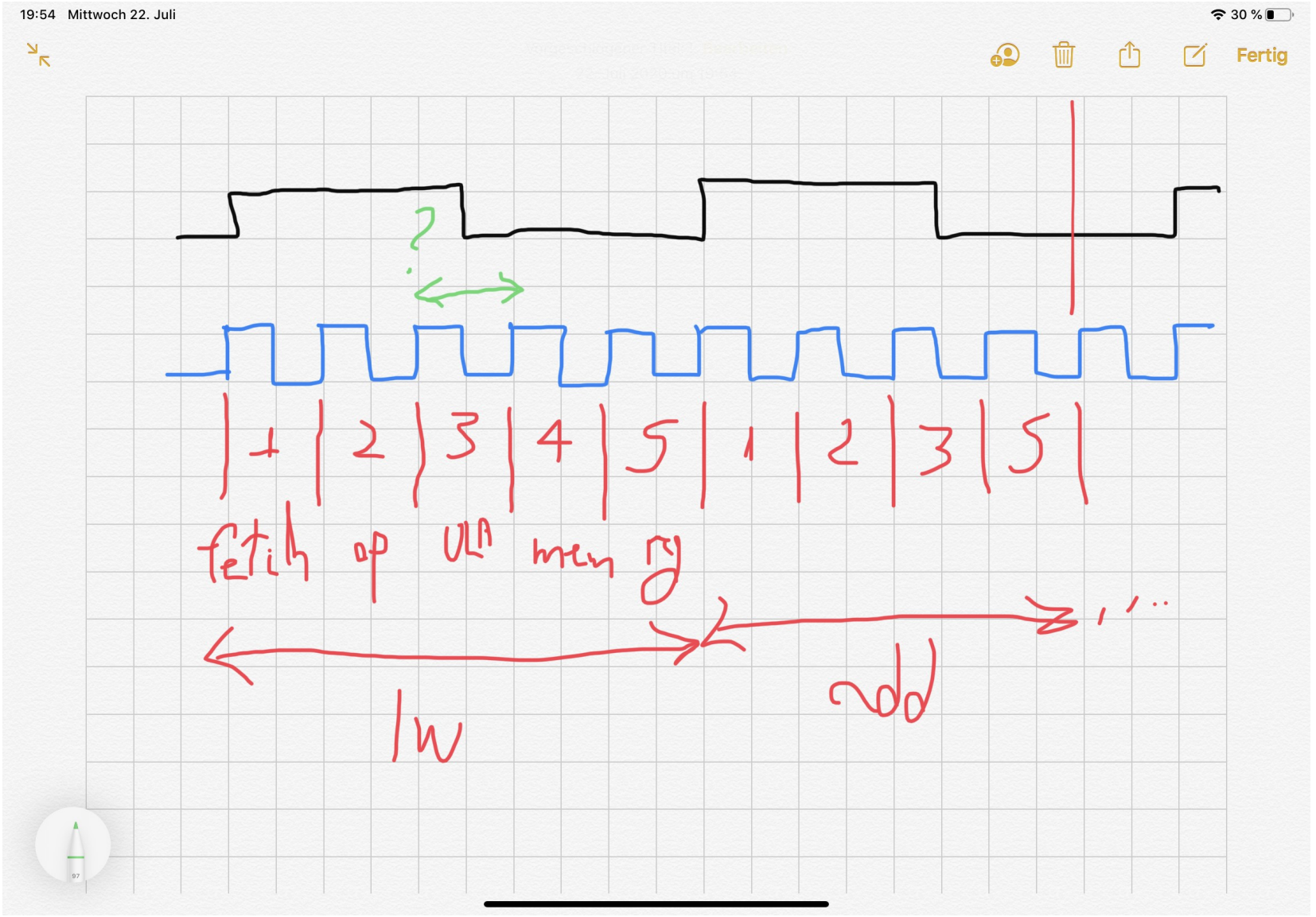
Dividir execução em partes



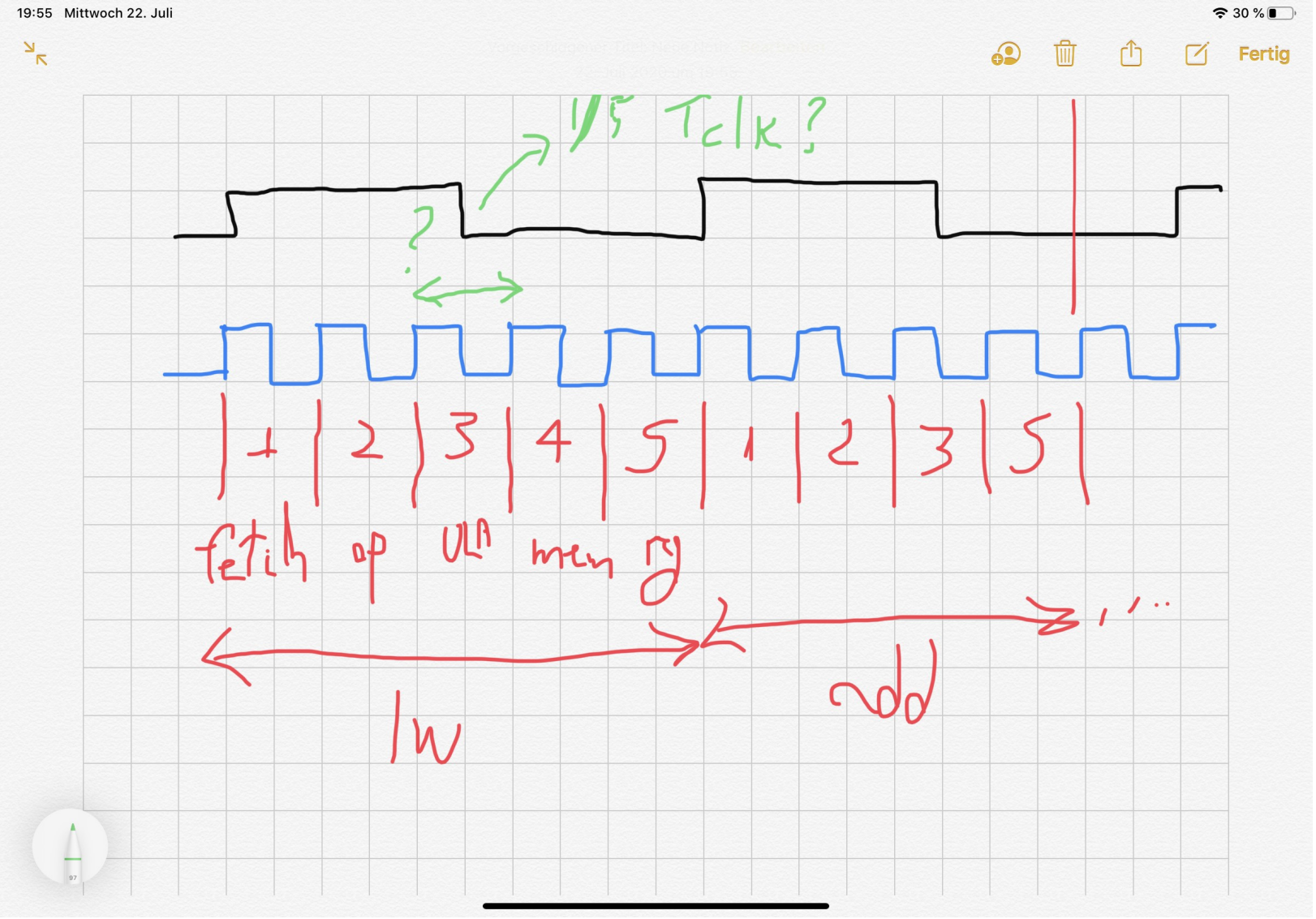
Só use as necessárias



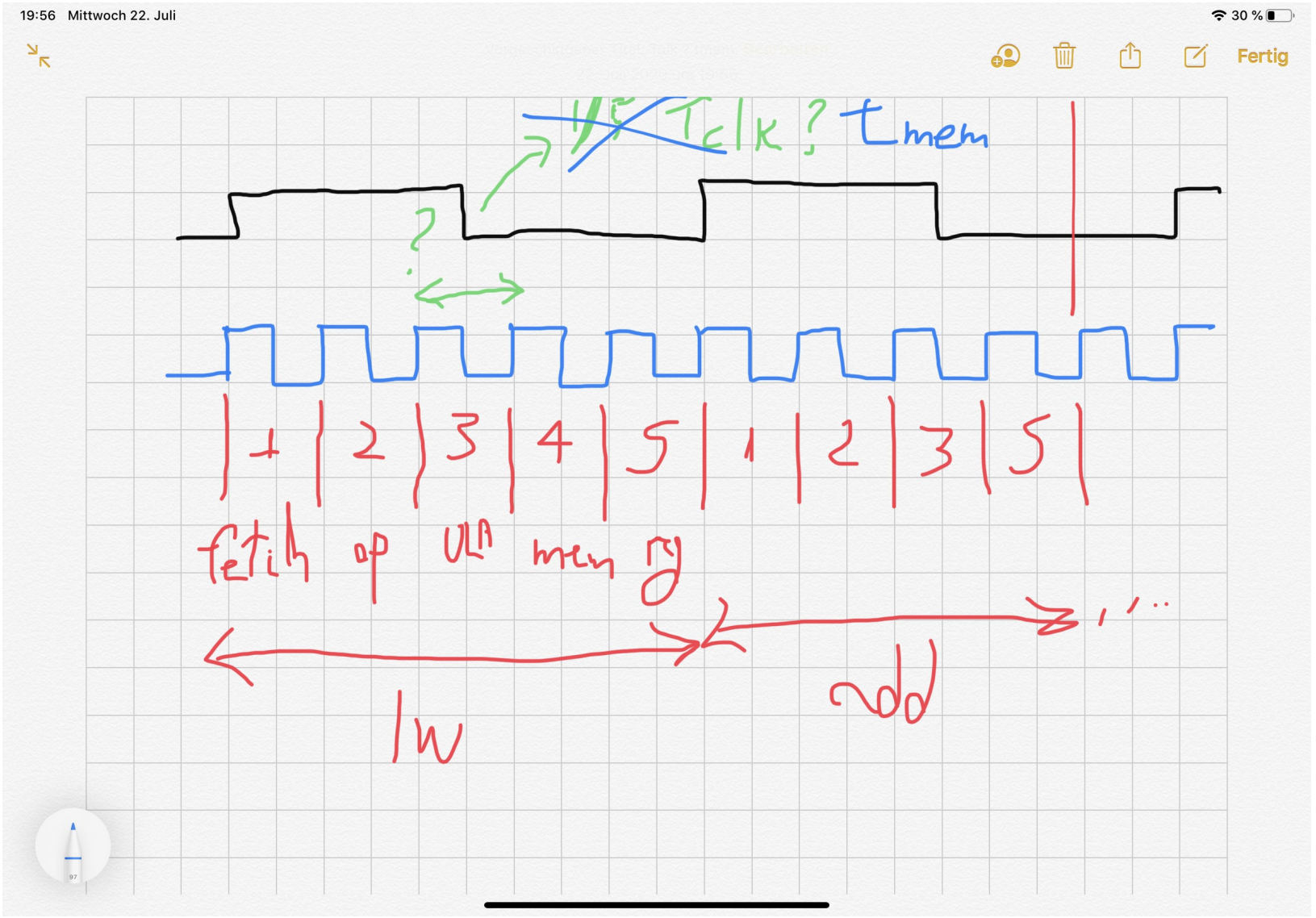
Período mínimo cada parte?



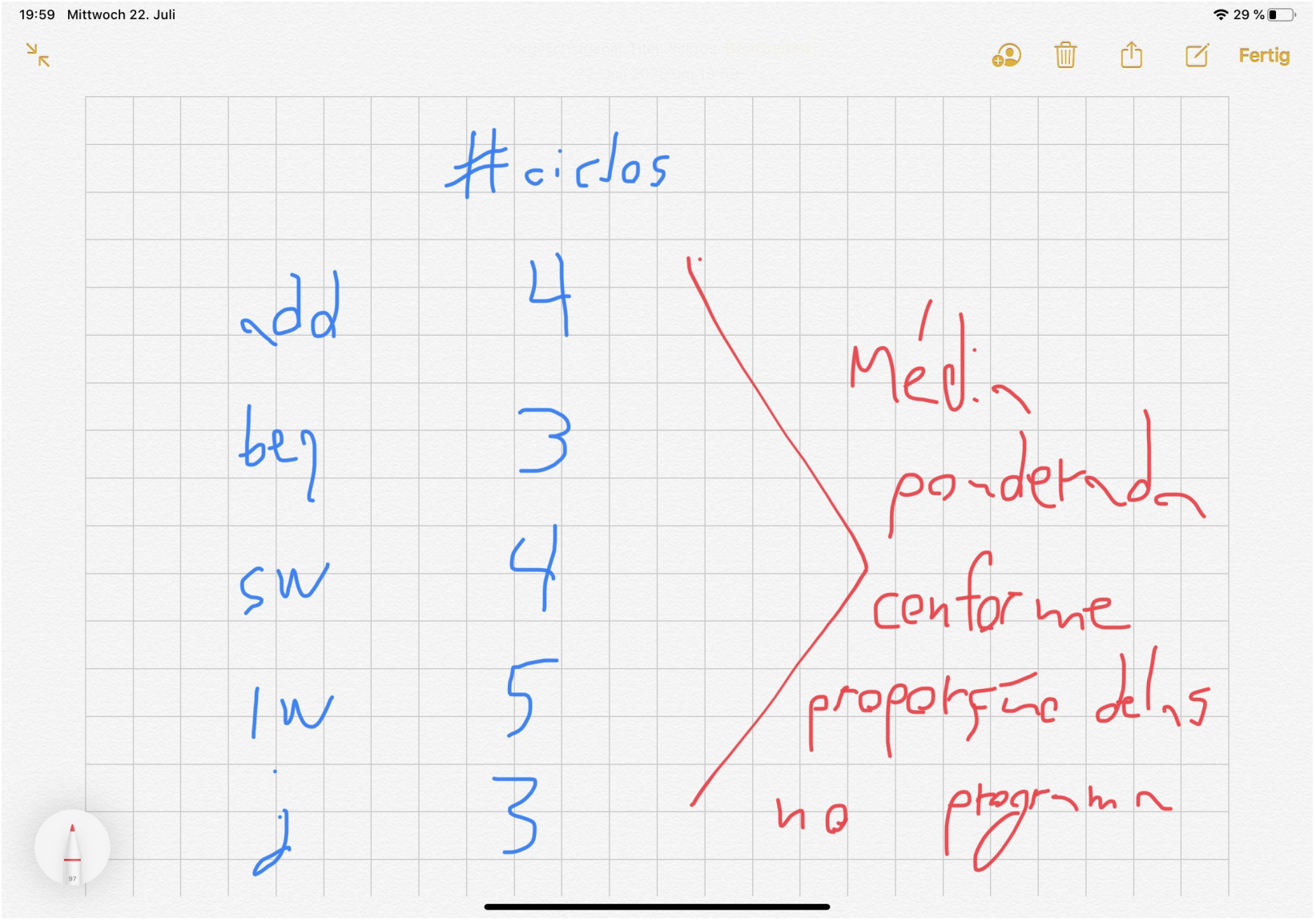
Seria 1/5 do período do monociclo?



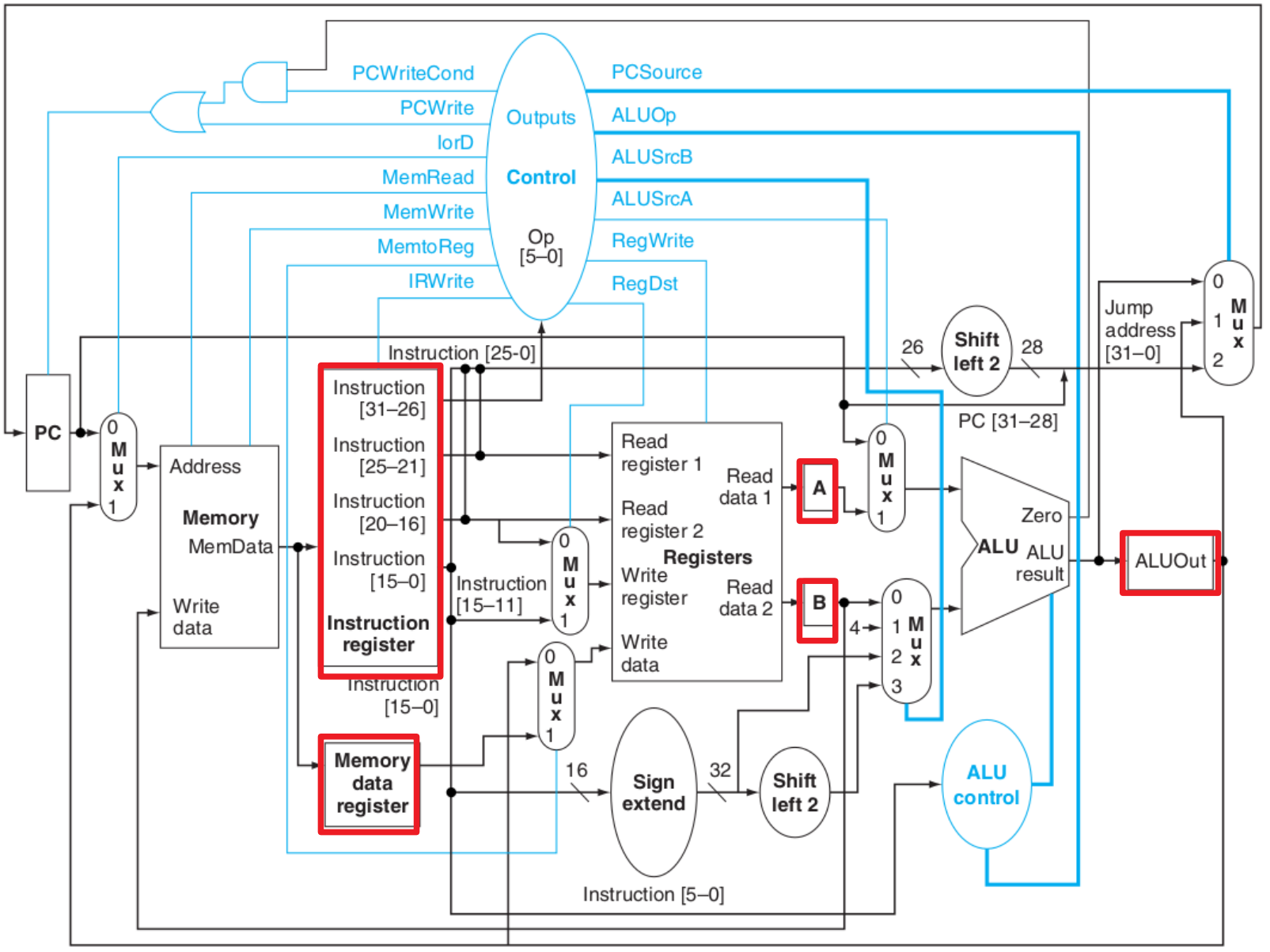
Precisa atender parte mais lenta



Ciclos médios por instrução



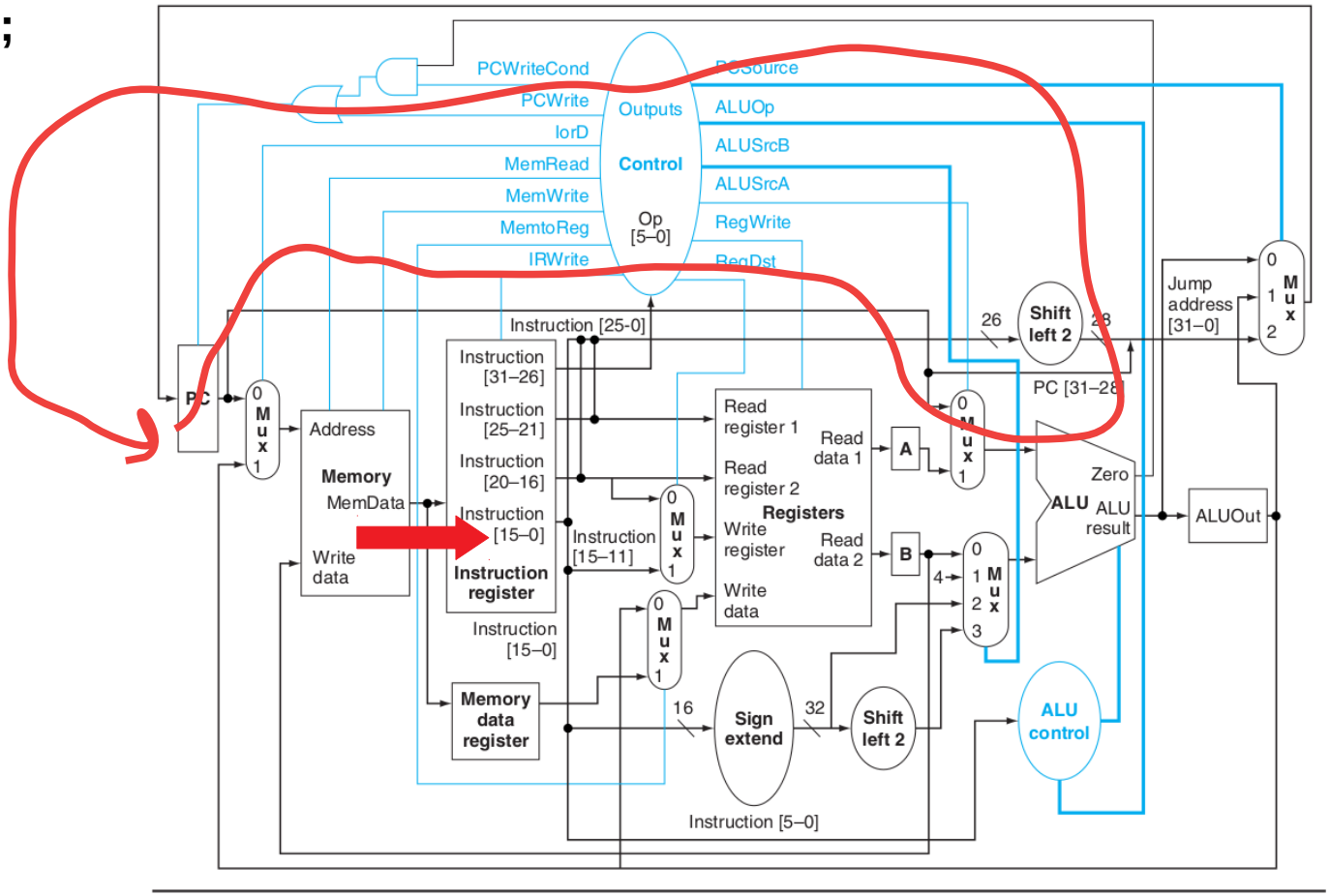
Multicycle datapath



Partes da execução

1 Instruction fetch

IR <= Memory[PC];
PC <= PC + 4;



No ciclo 1 a ULA estava livre, foi aproveitada para calcular PC+4 (veja que sumiu o somador que calculava isso)

A instrução foi lida da memória e guardada no IR

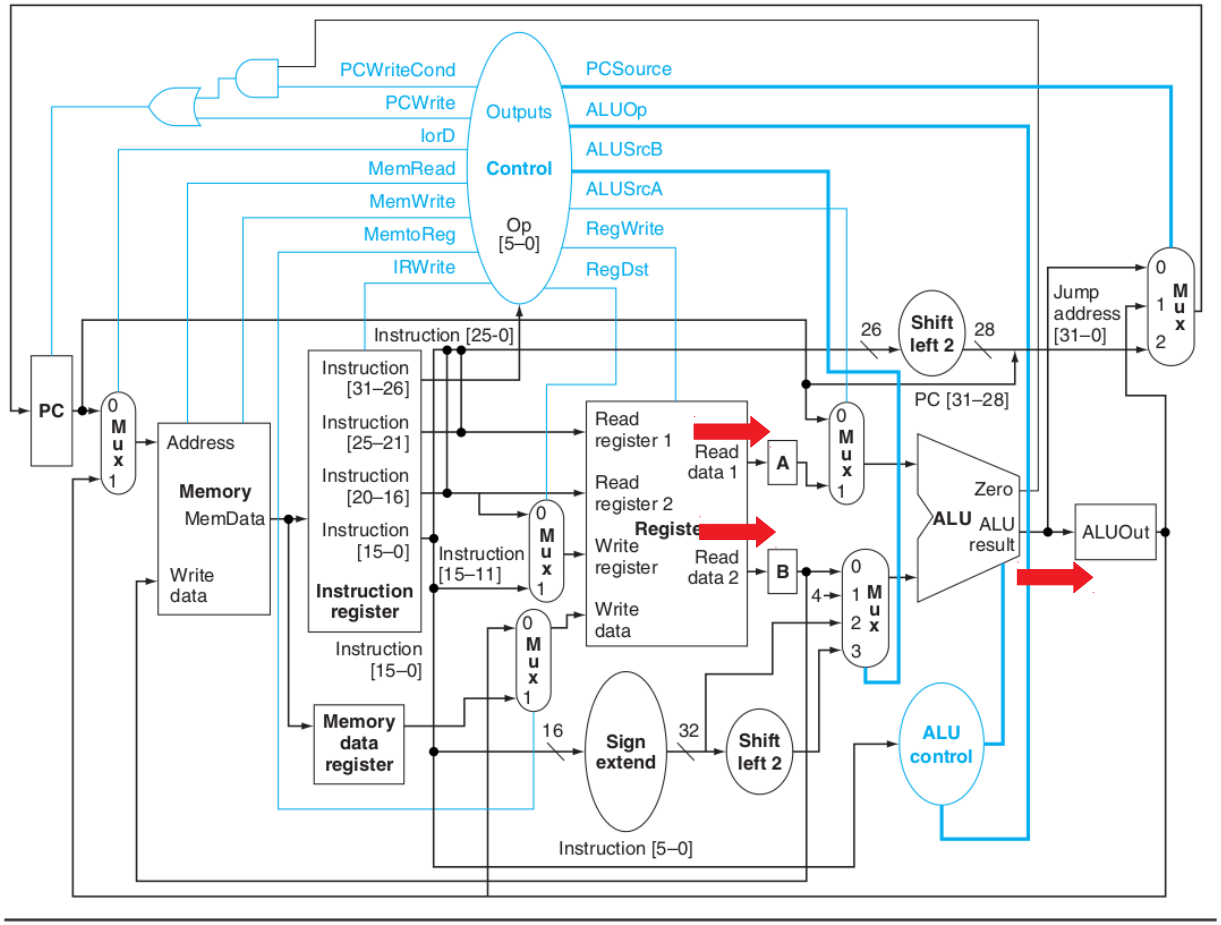
Partes da execução

2 Instruction decode and register fetch

A <= Reg[IR[25:21]];
B <= Reg[IR[20:16]];
ALUOut <= PC + (sign-extend (IR[15-0]) << 2);

Leu registradores A e B (mesmo que não use o B)

No ciclo 2 a ULA estava livre, foi aproveitada para calcular endereço de desvio do beq (veja que também sumiu o somador que calculava isso); se aparecer beq já está pronto; se não for, ao menos não atrasou nada por fazer isso; guardou esse resultado em ALUOUT

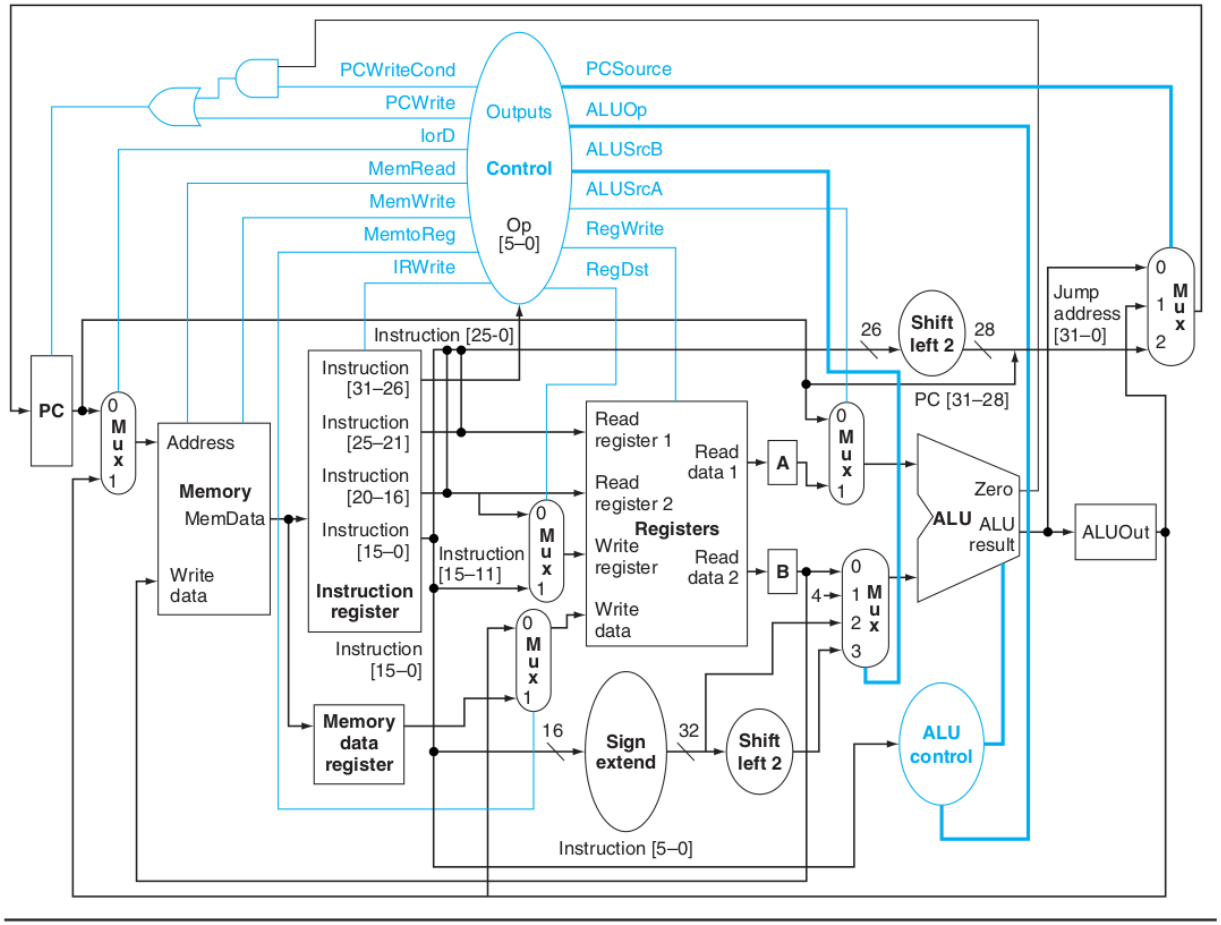


Partes da execução

3 Execution, memory address computation, or branch completion

Agora o que fazer depende se é instrução tipo R (lógica aritmética), ld/sw (precisa calcular endereço de memória), desvio condicional ou jump.

Vejamos nos próximos 4 slides ...



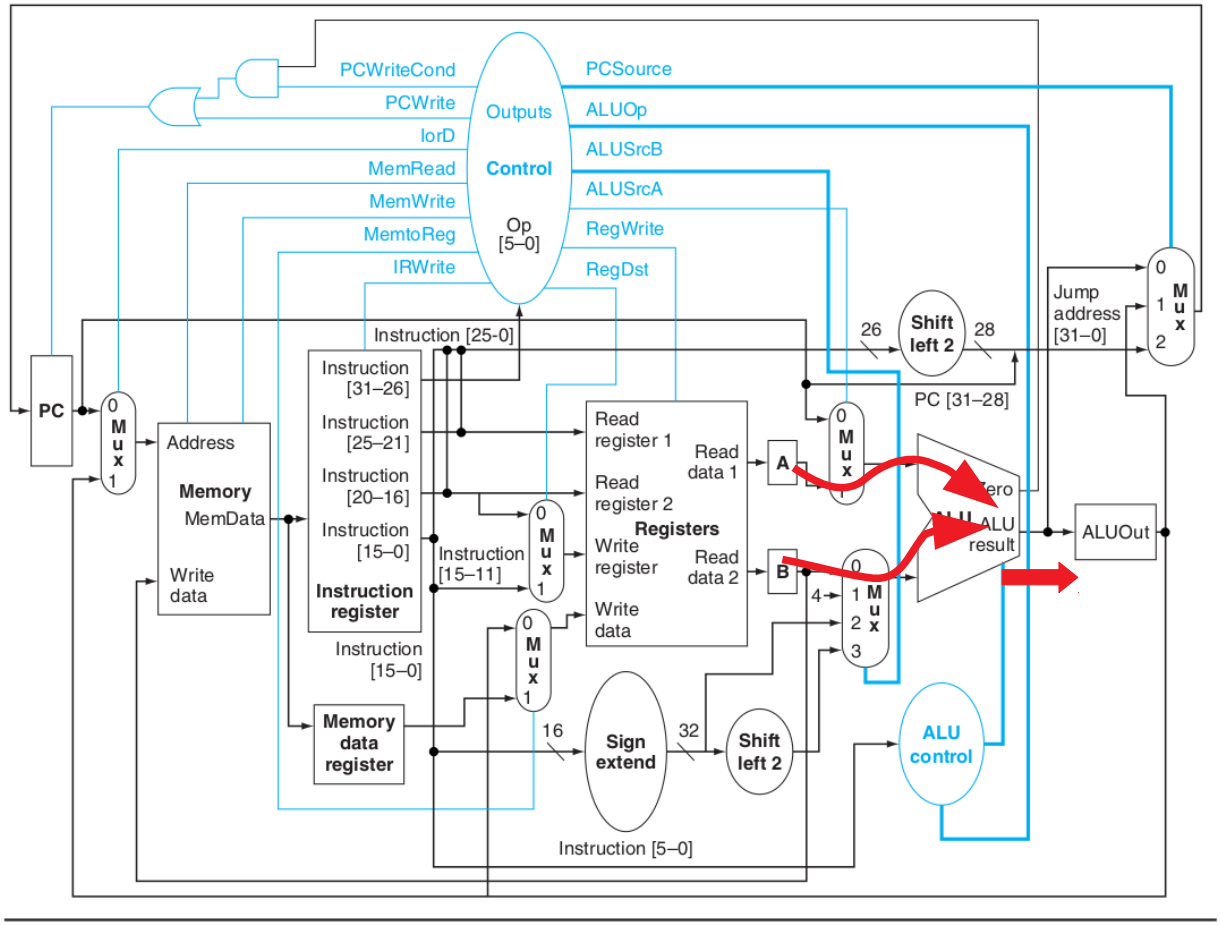
Partes da execução

3.1 Execution (R-type)

ALUOut <= A op B;

Simplesmente a ULA faz a operação necessária e armazenamos resultado no ALUOUT para ficar disponível para a próxima parte do processamento.

Veja que a ULA foi usada no ciclo 1, 2 e agora no 3



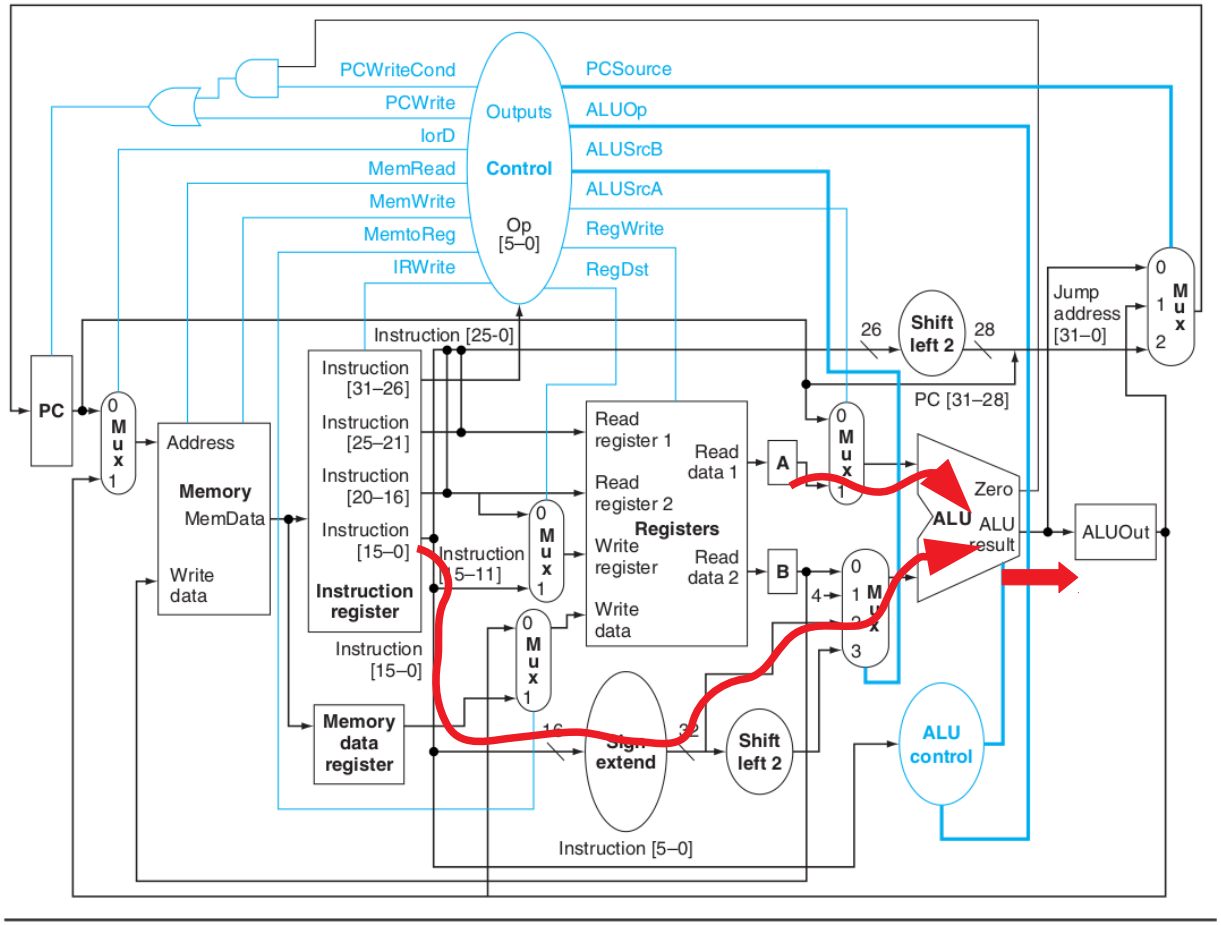
Partes da execução

3.2 Memory address computation (lw/sw)

ALUOut <= A + sign-extend (IR[15:0]);

Aqui a ULA calcula o endereço somando reg A e o imediato extendido para 32 bits, armazenando resultado no ALUOUT para ficar disponível para a próxima parte do processamento.

Veja que a ULA foi usada no ciclo 1, 2 e agora no 3



Partes da execução

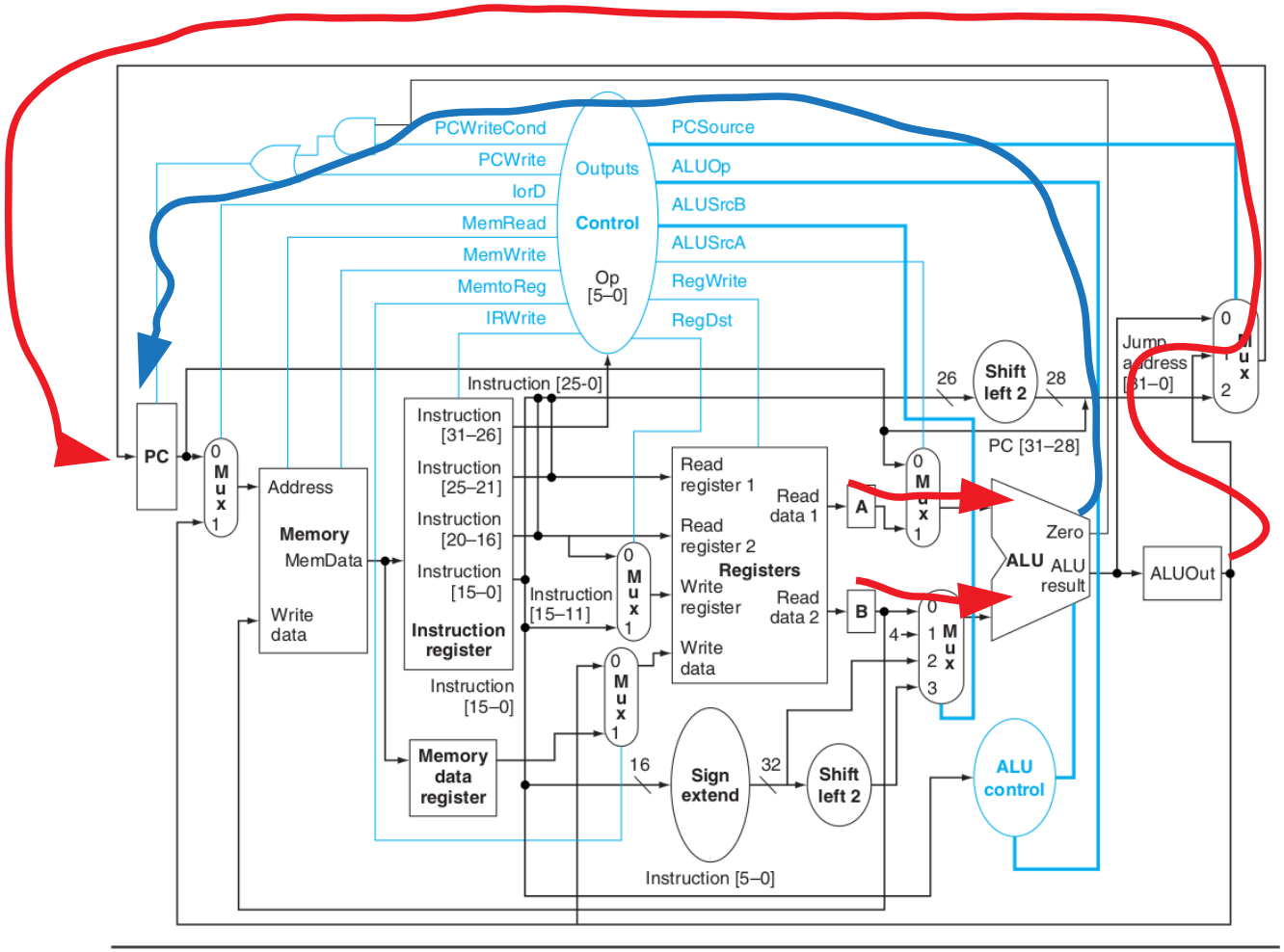
3.3 Branch (beq)

if (A == B) PC <= ALUOut;

Cuidado, preste atenção. Aqui a ULA calcula A-B para ver se os regs são iguais, o sinal de Zero vai ser utilizado para escolher entrada do mux que vai para o PC.

O endereço de desvio foi calculado no ciclo anterior e está disponível no ALUOUT, se for desviar carregamos o PC com o valor de ALUOUT.

Veja que a ULA foi usada no ciclo 1, 2 e agora no 3.



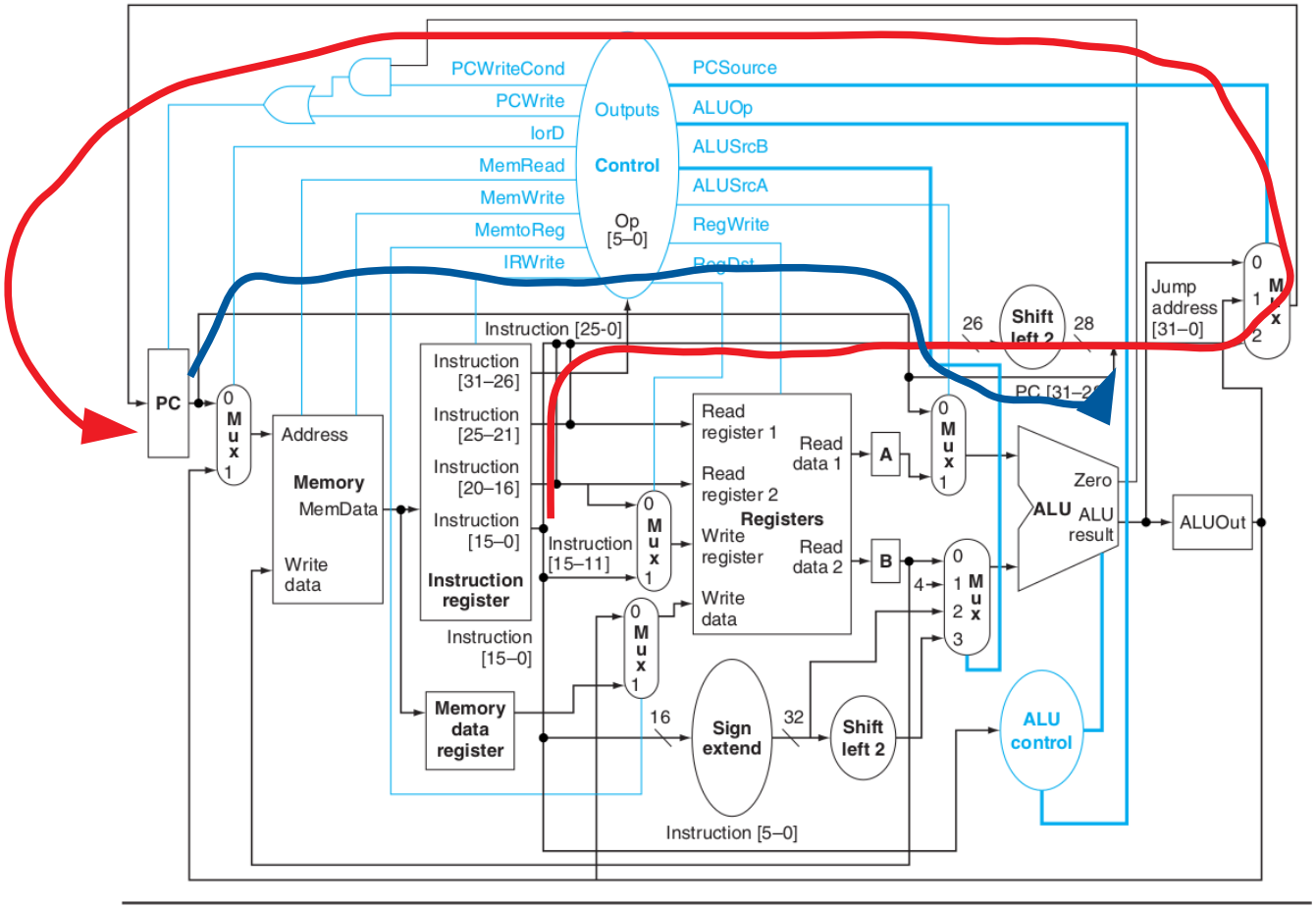
Partes da execução

3.4 Jump (j)

 # {x, y} is the Verilog notation for concatenation of bit fields x and y
PC <= {PC [31:28], (IR[25:0]],2'b00)};

O novo PC é formado pela concatenação dos 4 bits mais significativos do PC com os 26 do imediato da instrução e dois zeros ao final.

Veja que a ULA foi usada no ciclo 1, 2 e agora não foi usada no 3.

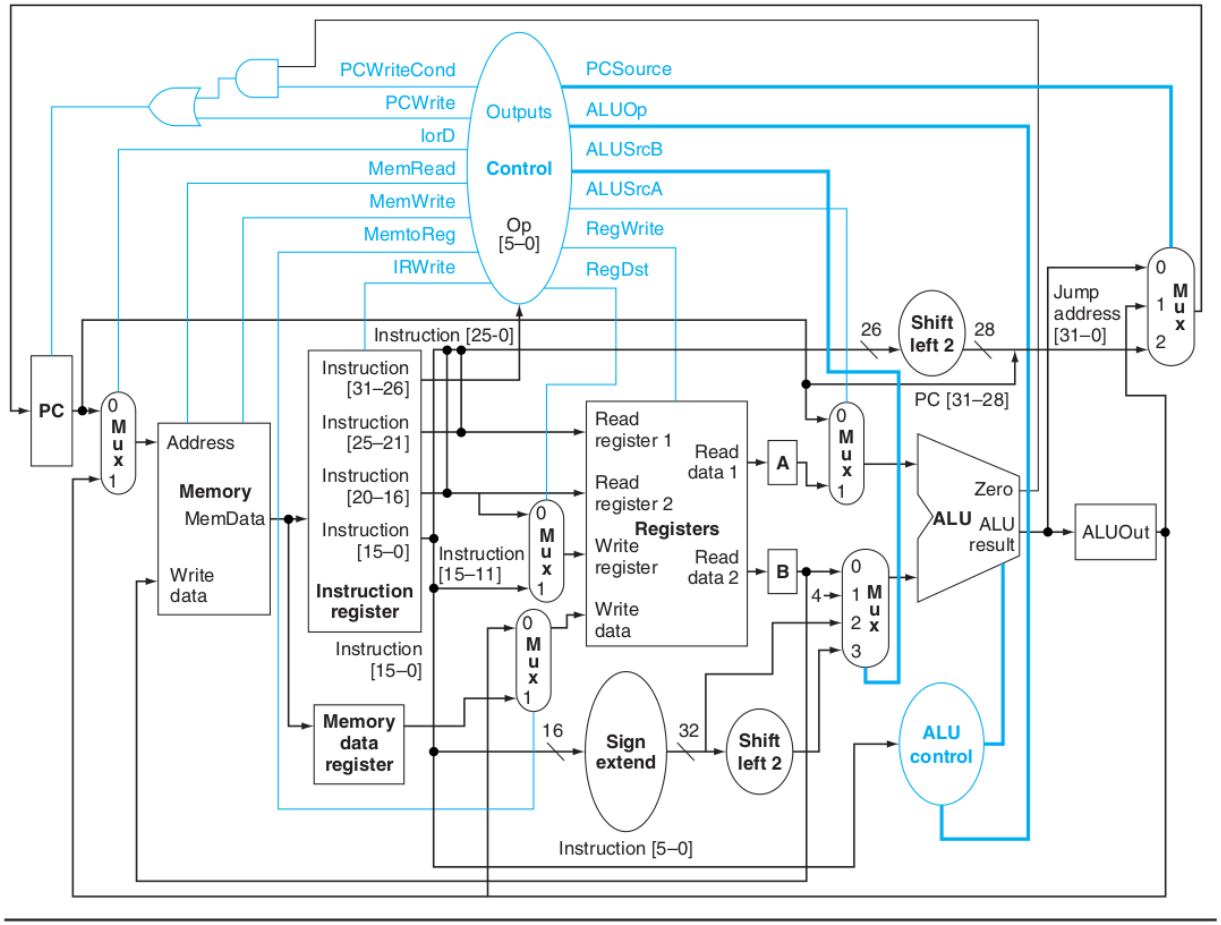


Partes da execução

4 Memory access or R-type instruction completion step

O que fazer depende se é instrução tipo R (lógica aritmética) ou lw/sw (ler ou escrever da memória)

Vejamos nos próximos 2 slides ...



Partes da execução

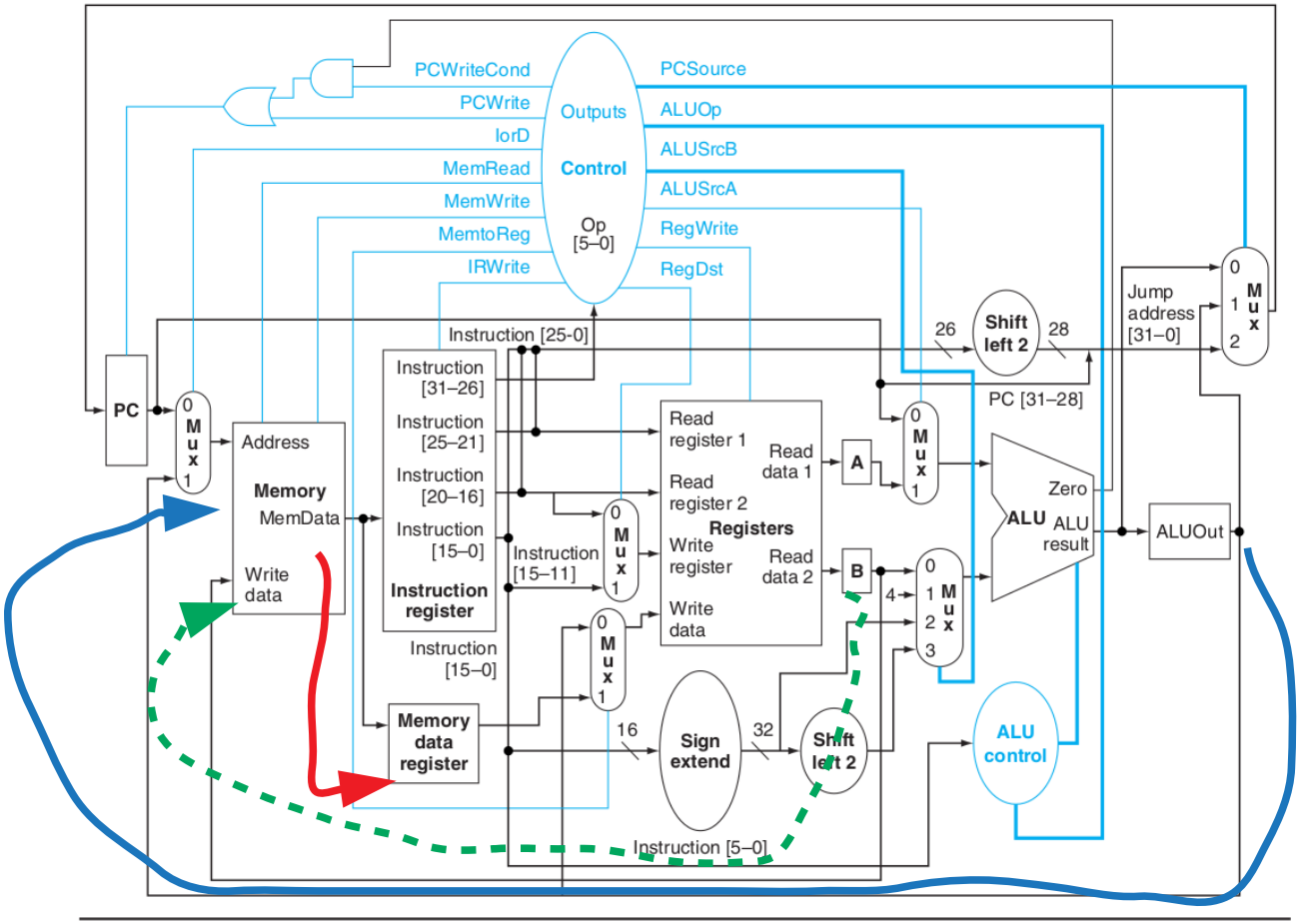
4.1 Memory access (lw/sw)

MDR <= Memory [ALUOut]; // se for lw
ou
Memory [ALUOut] <= B; // se for sw

Se for lw lê a memória para o MDR.

Se for sw guarda o B na memória.

Veja que para ambos casos o endereço foi calculado no ciclo anterior e estava em ALUOUT.



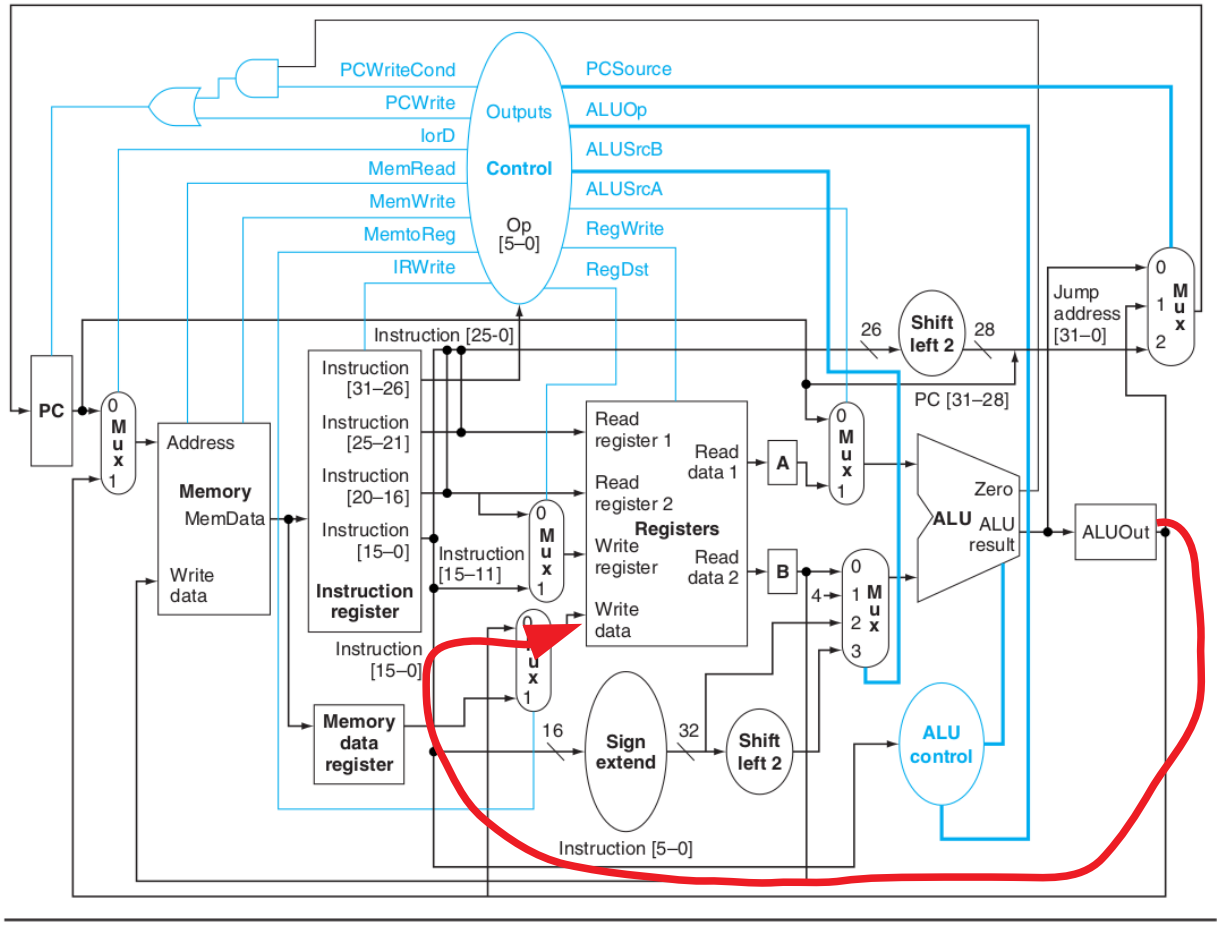
Partes da execução

4.2 R-type instruction completion (lógico-aritmética)

Reg[IR[15:11]] <= ALUOut;

O resultado calculado pela ULA é armazenado no banco de registradores.

Note que está em ALUOUT e não na saída direta da ULA.



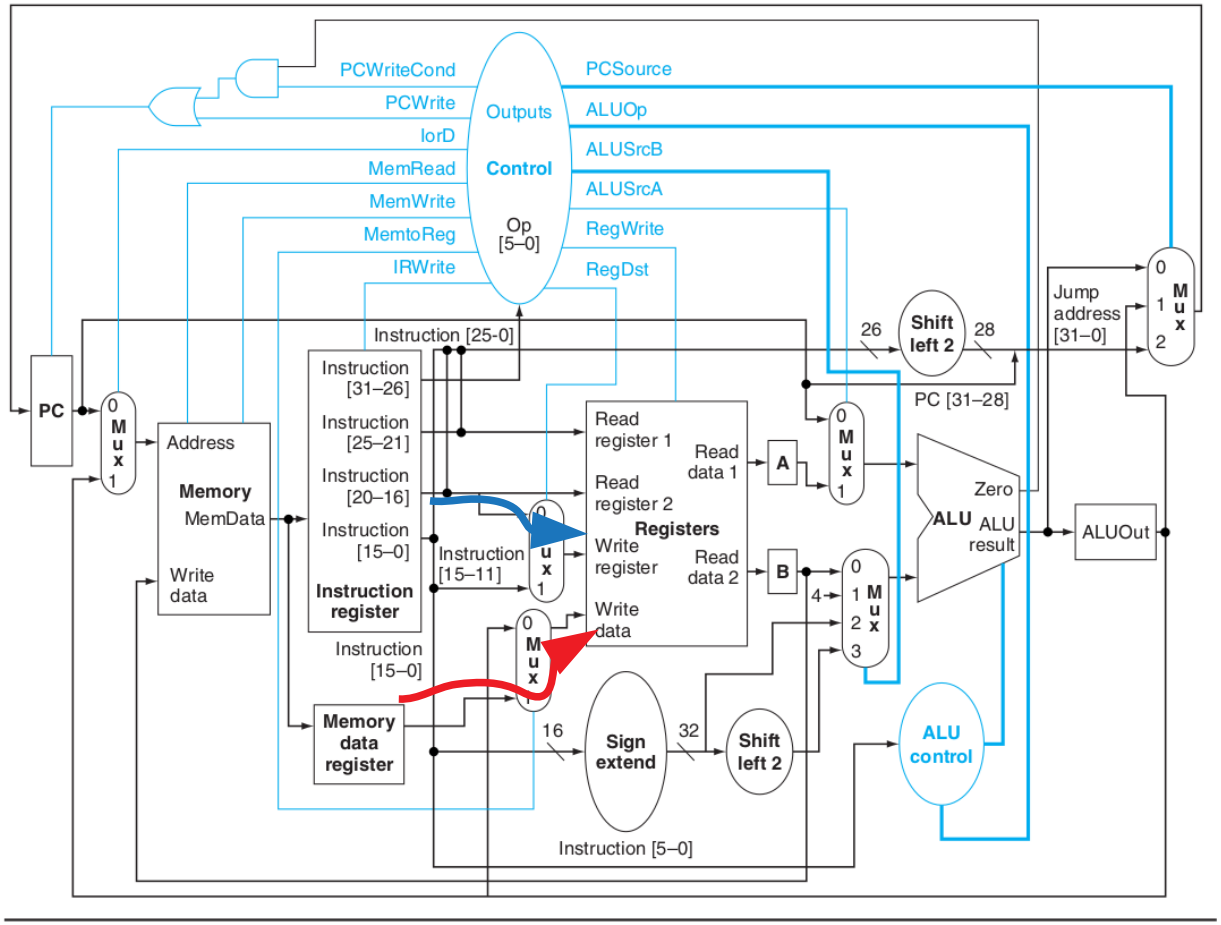
Partes da execução

5. Memory read completion step (lw)

Reg[IR[20:16]] <= MDR;

O valor lido da memória, que foi armazenado no ciclo anterior em MDR, é agora armazenado no banco de registradores.

Note que o endereço do registrador está dado por Rt, já que Rd não existe na instrução lw; seu campo foi ocupado pelo valor imediato.



Resumo

Step name	Action for R-type instructions	Action for memory-reference instructions	Action for branches	Action for jumps
Instruction fetch	IR <= Memory[PC] PC <= PC + 4			
Instruction decode/register fetch	A <= Reg [IR[25:21]] B <= Reg [IR[20:16]] ALUOut <= PC + (sign-extend (IR[15:0]) << 2)			
Execution, address computation, branch/jump completion	ALUOut <= A op B	ALUOut <= A + sign-extend (IR[15:0])	if (A == B) PC <= ALUOut	PC <= {PC [31:28], (IR[25:0]),2'b00)}
Memory access or R-type completion	Reg [IR[15:11]] <= ALUOut	Load: MDR <= Memory[ALUOut] or Store: Memory [ALUOut] <= B		
Memory read completion		Load: Reg[IR[20:16]] <= MDR		

Créditos

Figuras de Patterson&Hennessy 2018, 2004 – Morgan Kaufmann

CI1212 Arquitetura de Computadores

Prof. Eduardo Todt

todt@inf.ufpr.br

Período Especial 2020